

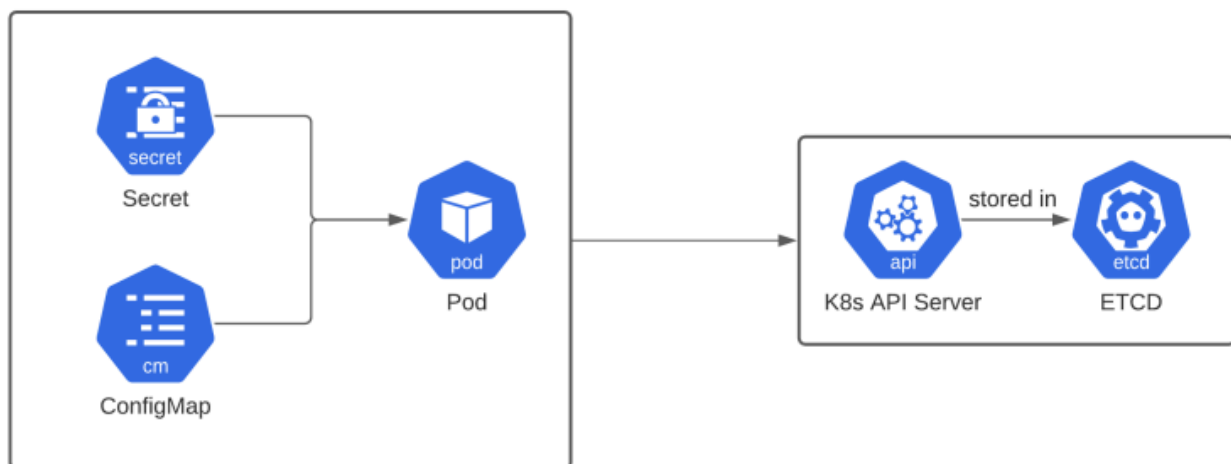
📦 Kubernetes ConfigMaps & Secrets — Complete Teaching Guide

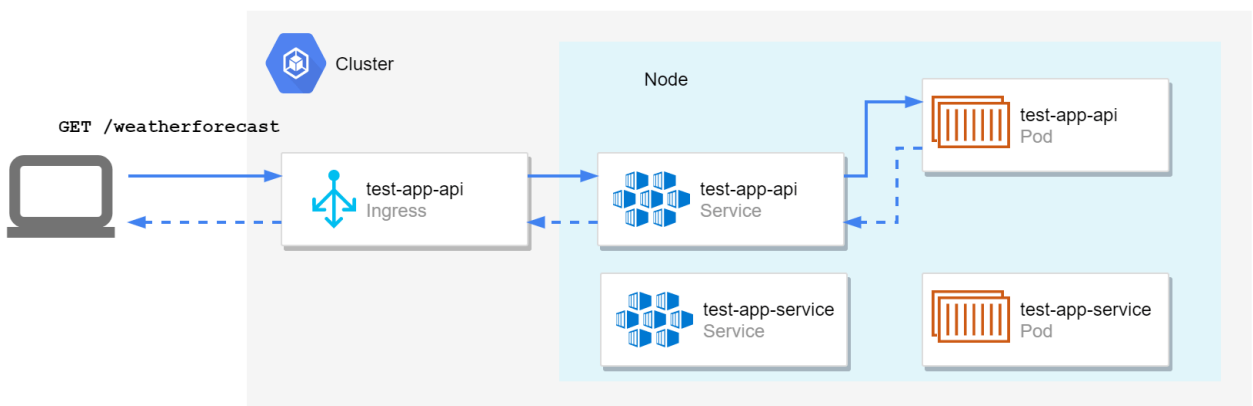
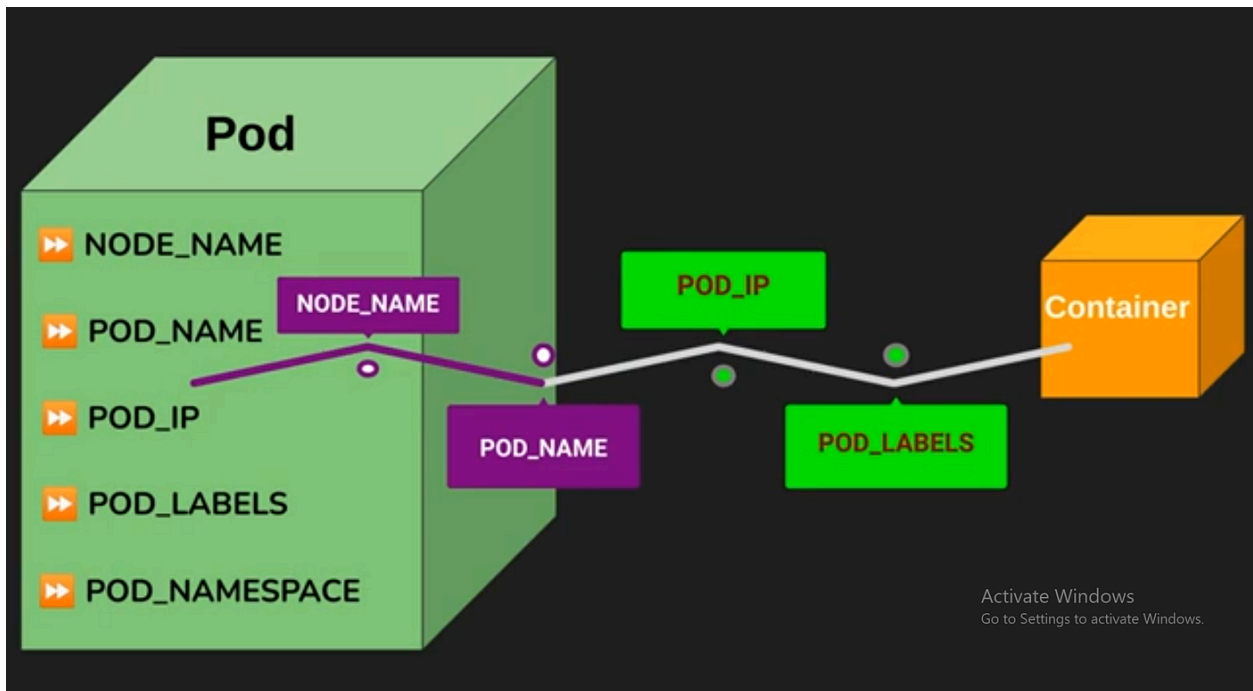
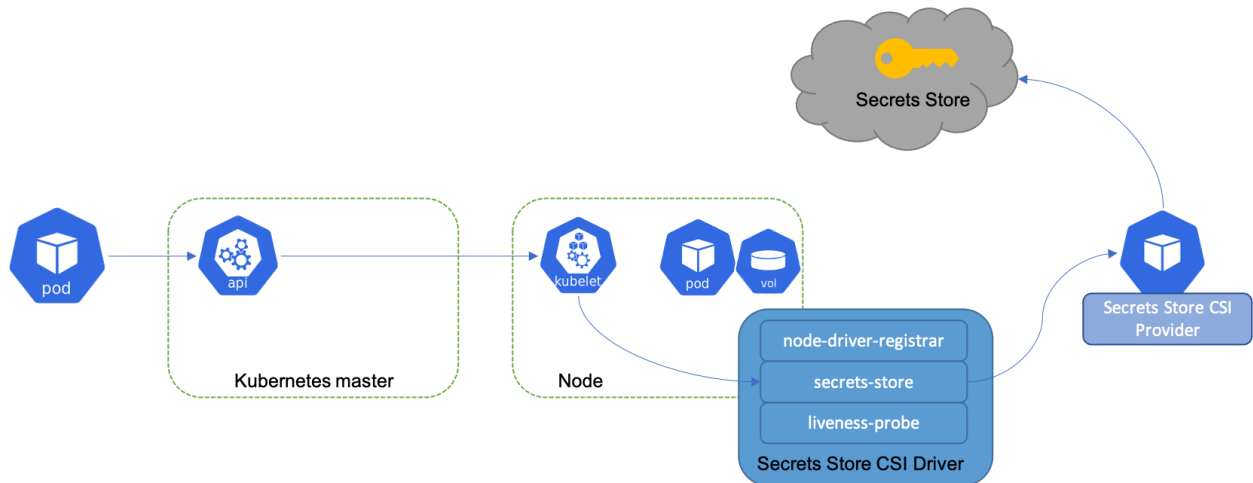
In Kubernetes, **ConfigMaps** and **Secrets** are fundamental building blocks for managing application configuration securely and flexibly.

This guide explains:

- What they are
 - Why they matter
 - How Kubernetes uses them
 - Production best practices
-

🧠 Big Picture: Why Do We Need Them?





Modern applications must:

- Run in multiple environments (dev, staging, prod)
- Use different credentials per environment
- Avoid hardcoding configuration inside images
- Rotate secrets without rebuilding containers

ConfigMaps and Secrets solve this.

1 What Is a ConfigMap?

Definition

A **ConfigMap** is a Kubernetes API object used to store **non-sensitive configuration data** as key-value pairs.

Purpose

- Decouple configuration from container images
 - Avoid rebuilding images for small config changes
 - Enable environment-specific configurations
-

Example Use Cases

- Log levels → `LOG_LEVEL=debug`
 - Feature flags → `FEATURE_X_ENABLED=true`
 - Service endpoints → `DB_HOST=db-service`
 - Application mode → `APP_MODE=production`
-

⚠ Important Notes

- ❌ Not encrypted by default
 - ✅ Can store multiple key-value pairs
 - ✅ Can be consumed as:
 - Environment variables
 - Files mounted inside containers
-

2 What Is a Secret?

📖 Definition

A **Secret** is a Kubernetes object used to store **sensitive data**, such as credentials or tokens.

🎯 Purpose

- Protect sensitive information
 - Separate secrets from container images
 - Enable secret rotation without image rebuild
-

🔑 Example Use Cases

- DB_PASSWORD
 - PAYMENT_API_KEY
 - tls.crt, tls.key
 - OAuth tokens
-

🔍 Important Security Details

- Stored as **Base64-encoded** in YAML/JSON
 - Can be **encrypted at rest** (if configured)
 - Access controlled via **RBAC** (Role-Based Access Control)
 - Should be mounted as **read-only volumes** when possible
-

3 ConfigMap vs Secret — Comparison

Feature	ConfigMap	Secret
Data Type	Non-sensitive config	Sensitive credentials
Storage	Plain text	Base64-encoded
Encryption at Rest	✗ Not default	✓ Supported
Access Control	Optional	Strongly recommended
Volume Mount	✓ Yes	✓ Yes (read-only recommended)
Use Case	App config, feature flags	Passwords, tokens, TLS

🧠 Simple Rule

ConfigMaps = Configuration
Secrets = Credentials

4 How Kubernetes Injects Them Into Pods

Kubernetes allows two main injection methods:

A As Environment Variables

envFrom:

- configMapRef:
 name: my-config
- secretRef:
 name: my-secret

What Happens:

- Each key becomes an environment variable
- App reads values like:
 - `$LOG_LEVEL`
 - `$DB_PASSWORD`

B As Mounted Volumes

volumes:

- name: config-vol
 configMap:
 name: my-config
- name: secret-vol
 secret:
 secretName: my-secret

What Happens:

- Each key becomes a file
- Apps read from filesystem
- Common for:
 - TLS certificates

- Config files
 - Structured YAML/JSON configs
-

Production Considerations

Separation of Concerns

- ConfigMaps → non-sensitive settings
- Secrets → sensitive credentials

Never mix them.

Environment Flexibility

Create separate objects per environment:

- `app-config-dev`
- `app-config-staging`
- `app-config-prod`

Same container image — different configurations.

Secret Rotation

Typical workflow:

1. Update Secret object
2. Trigger rolling restart of Deployment
3. New Pods consume updated secret

No image rebuild required.

Security Best Practices

- Enable encryption at rest
- Use RBAC to restrict access
- Mount Secrets as read-only
- Rotate credentials regularly
- Avoid exposing secrets via logs

Dynamic Updates

- ConfigMaps mounted as volumes can update automatically
- Environment variables require Pod restart
- Production systems often use rollout restarts

Typical Production Workflow

1. Create ConfigMap
2. Create Secret
3. Deploy application referencing them
4. Update ConfigMap for config changes
5. Update Secret for credential rotation
6. Perform rolling updates to apply changes
7. Monitor application behavior

This enables:

- Zero-downtime updates
 - Secure credential management
 - CI/CD flexibility
-

7 Why ConfigMaps & Secrets Matter

CI/CD Efficiency

- Same image across environments
 - Configuration managed externally
 - Faster deployments
-

Security

- Credentials isolated
 - Access controlled via RBAC
 - Optional encryption at rest
-

Multi-Environment Support

- Dev, staging, prod share image
 - Each environment has unique configuration
-

Operational Flexibility

- Update configuration without rebuild
 - Rotate credentials without downtime
 - Improve incident response speed
-

Teaching Summary

Concept

Core Idea

ConfigMap	Externalize non-sensitive config
Secret	Securely store sensitive data
Injection	Environment variables or volumes
Production Use	Environment separation + secret rotation

Final Conceptual Insight

In Kubernetes:

- **Images contain application code**
- **ConfigMaps contain behavior configuration**
- **Secrets contain trust and credentials**

This separation is what makes Kubernetes powerful for modern, secure, scalable systems.

If you'd like, I can also create:

-  A hands-on lab version
-  Instructor notes with discussion questions
-  A security-focused deep dive
-  A real-world enterprise architecture example